

class 06

Maasfa Shaikh

Question 1: Write a function add() to add some numbers together. Make sure to add comment(s) to the function explaining why you do things.

a. Your first version of the function should add two input numbers together; e.g. add(4, 7) should return 11. [1 pt] b. Your second version of the function should add any numbers together that the user inputs; e.g. add(1, 2, 3, -4) should return 2. [1] pt

```
#answer 1A.
```

```
add_2 <- function(x,y) {  
  x+y  
}
```

```
add_2(y=5, x=8)
```

```
[1] 13
```

```
#making a function to add two variables and testing it out
```

```
#Answer 1B: you can put ... to tell the signal that you can put in additional values
```

```
#the first method is used if you want to make a vector first
```

```
add_multi <- function (x, ...){  
  v <-c(x,...)  
  sum(v)  
}
```

```
#easier way to do this is just to do sum directly
```

```
add_multi_sum <- function (x,...) {
```

```

    sum(x,...)
}

add_multi(1,2,3,4,-2)

```

```
[1] 8
```

```

#to add something more to an existing function, you can extract function using "code". You s

add_multi_sum <- function(x, ...) {
  sum(x,...)
}

```

Question 2: Write a function `generate_dna()` that generates a random DNA sequence of a length input by the user; e.g. `generate_dna(11)` may return “ATTTAAGGCTG”. Make sure to add comments to the function. Play with functions `sample()` and `paste()` in your R-Studio Console to see how they can help here. [1 pt]

```

generate_dna <- function(n) {
  nucs <- c("A", "G", "T", "C")
  #making the vector "nucs" with the nucleotides
  nucs_vector <- sample(nucs, n, replace=TRUE)
  #using sample to pull out random nucleotides
  paste(nucs_vector, collapse = "")
  #generating DNA string using the paste and collapse function
}

#testing my function
generate_dna(10)

```

```
[1] "TTATACGAGC"
```

Question 3: Write a function `generate_protein()` that generates a random peptide/protein sequence of a length input by the user; e.g. `generate_protein(6)` may return “WQRTAG”. Make sure to use the single-letter code for all 20 amino acids and use no other letters (see list of amino acids at the bottom of the page); make sure that individual amino acids can appear more than once. Make sure to add comments to the function. [1]pt

```
#starting with assigning the the function a name
generate_protein <- function(n) {
  #defined the value for the function
  amino_acids <- c("A", "R", "N", "D", "C", "E", "Q", "G", "H", "I", "L", "K", "M", "F", "P")
  #made a vector with all the amino acid letter codes
  amino_acids_vector <- sample(amino_acids, n, replace=TRUE)
  #used sample to pull out random amino acids from the amino_acids vector
  #set replace to true so the amino acids can be reused
  paste(amino_acids_vector, collapse="")
  #used paste and collapse so that there is a string of amino acids with nothing in between
}

generate_protein(10)
```

```
[1] "TMICPYGTML"
```

Question 4: Use your `generate_protein()` function to generate peptides of lengths 5, 7, 9, and 11 amino acids. Take advantage of the base R function `apply()` for this. [1 pt]

```
#listed out what the sample size I want by assigning n a list of 5,7,9, and 11
n <- list(5,7,9,11)
#used apply to make a simplify the characters in the function generate_protein and pull out a
apply(n, generate_protein)
```

```
[1] "VMMQA"          "FDWHKIW"        "WSSIIVDKS"      "NKSEWSNTEPA"
```

Question 5: Take each of your peptides from Q4 and run a BLASTP search (<https://blast.ncbi.nlm.nih.gov/Blast.cgi>) selecting the Non-redundant protein sequences (nr) database and homo sapiens (taxid: 9606) organism. For each search, before clicking BLAST check the Show results in a new window tab next to the BLAST button to easily allow you to run the blast search for all four peptides at the same time in four different tabs. Once each search is done, for the top aligned protein for each peptide, list the percent identity (Per. Ident) times coverage (Query Cover) (for example, in the example below, $100\% \times 85.71\% = 1.00 \times 0.8571 \times 100\% = 85.71\%$); make sure to type out your calculations in your Quarto document [note: if any of your values are below 30% you are probably miscalculating something]. Upload your numbers for each peptide (length (aa) and max identity (%)) into the shared class excel data sheet (see Data Sheet link

in the Canvas assignment). Also, for each datapoint, add your initials in the “Your_Initials” column of the data sheet. Before adding your initials, scan through other used initials; if anyone has the same initials as you, modify your initials to make them unique (your initials will be used below to identify your specific data points in a ggplot graph). [1 pt]

Answer 5: sequence 1: (n=5) NGNQR percent identity= 100.00%, query coverage: 100%

```
(1.0 * 1.0 * 100)
```

```
[1] 100
```

sequence 2: (n=7) HRNPCYH Percent identity= 85.71%, Query coverage: 100%

```
0.8571 * 1.00 * 100
```

```
[1] 85.71
```

sequence 3: (n=9) YDTLLGDQQ percent identity: 100.0%, query coverage: 78%

```
1.0 * 0.78 * 100
```

```
[1] 78
```

sequence 4: (n=11) PHHSFGWENDG Percent identity: 70.0%, query coverage: 91%

```
0.7 * 0.91 * 100
```

```
[1] 63.7
```

Question 6: Once there is a good amount of data in the shared excel sheet, export the sheet as a CSV UTF-8 file (Export option under the File tab) and save the file to your class working folder (i.e. the folder in which you are currently generating a Quarto document). In R-Studio, upload the csv file into a data frame using the command `peptide_df <- read.csv(“your_filename”)`. Use the `head()` function to list the first 10 rows (note that the default for `head()` is 6) of the `peptide_df` dataframe as a test that everything looks good. [1 pt]

```
#uploaded the saved excel sheet as a dataframe
peptide_df <- read.csv("BIMM143_peptide(Sheet1).csv")
#extracted the first 10 rows
head(n=10, peptide_df)
```

	Length_aa	Max_Identity_perc	Your_Initials	X	X.1	X.2	X.3	X.4	X.5	X.6	X.7	X.8
1	5	100.00	JLA	NA	NA	NA	NA	NA	NA	NA	NA	NA
2	7	85.71	JLA	NA	NA	NA	NA	NA	NA	NA	NA	NA
3	9	66.85	JLA	NA	NA	NA	NA	NA	NA	NA	NA	NA
4	11	55.00	JLA	NA	NA	NA	NA	NA	NA	NA	NA	NA
5	5	80.00	SKB	NA	NA	NA	NA	NA	NA	NA	NA	NA
6	7	85.71	SKB	NA	NA	NA	NA	NA	NA	NA	NA	NA
7	9	66.85	SKB	NA	NA	NA	NA	NA	NA	NA	NA	NA
8	11	64.00	SKB	NA	NA	NA	NA	NA	NA	NA	NA	NA
9	5	100.00	KET	NA	NA	NA	NA	NA	NA	NA	NA	NA
10	7	85.71	KET	NA	NA	NA	NA	NA	NA	NA	NA	NA
	X.9	X.10	X.11	X.12	X.13	X.14	X.15	X.16	X.17	X.18		
1	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA		
2	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA		
3	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA		
4	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA		
5	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA		
6	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA		
7	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA		
8	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA		
9	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA		
10	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA		

Question 7: Use ggplot2 to generate a box plot with Length_aa on the x-axis and Max_Identity_perc on the y-axis. Since the box plot will require categorical values for Length_aa, you may have to specify the x-axis as as.factor(Length_aa). Feel free to pretty up the plot by adding your own x/y-axis labels, graph title, etc using the labs() function and/or using your favorite theme using a theme() function (see the data-visualization_ggplot.pdf cheat sheet in Canvas for options). Optional: you will reuse this plot a few times below, so your subsequent coding will be simpler if you assign your box plot to an object such as 'p' (p <- ggplot(peptide_df) +....etc). [1 pt]

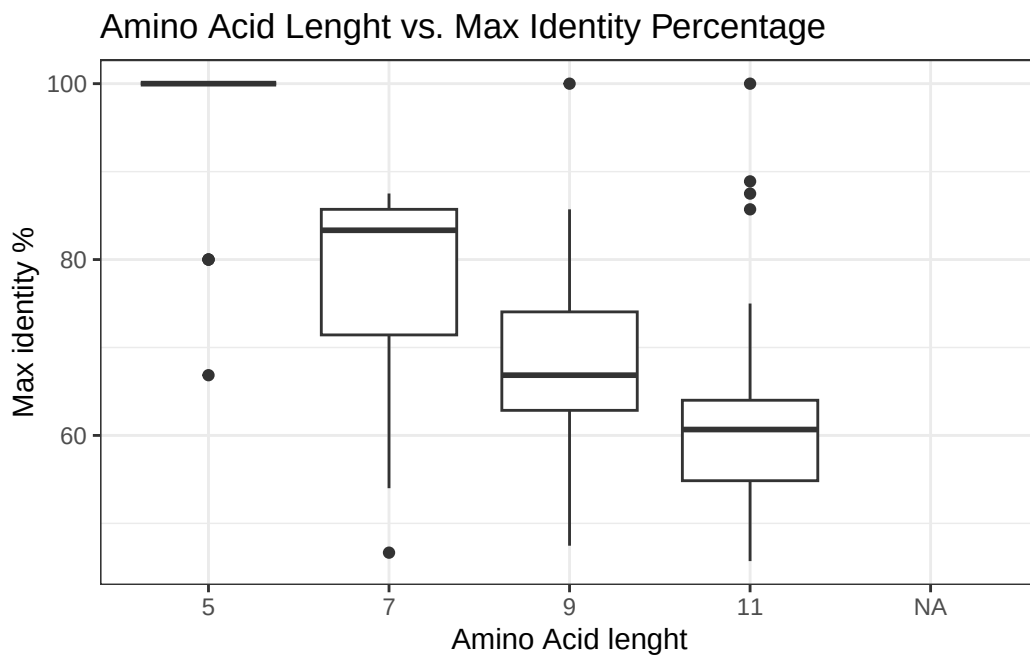
```
library(ggplot2) #loading in ggplot2 again
#creating boxplot with x and y axis
ggplot(peptide_df, aes(factor(Length_aa), (Max_Identity_perc)))+
```

```

geom_boxplot()+
#assigning labels to graph
labs(x= "Amino Acid lenght", y="Max identity %", title= "Amino Acid Lenght vs. Max Identity")
#adding a theme to graph
theme_bw()

```

Warning: Removed 4 rows containing non-finite outside the scale range (`stat\_boxplot()`).

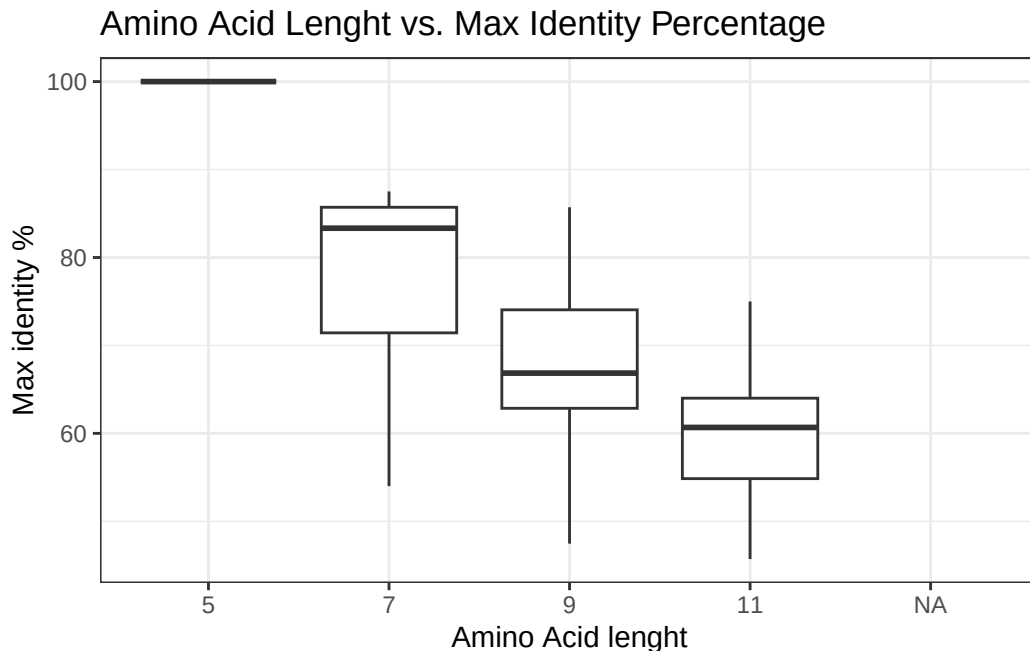


```

#making the graph an object
peptide_plot<- ggplot(peptide_df, aes(factor(Length_aa), (Max_Identity_perc)))+
  geom_boxplot(outliers=FALSE)+
  labs(x= "Amino Acid lenght", y="Max identity %", title= "Amino Acid Lenght vs. Max Identity")
  theme_bw()
peptide_plot

```

Warning: Removed 4 rows containing non-finite outside the scale range (`stat\_boxplot()`).

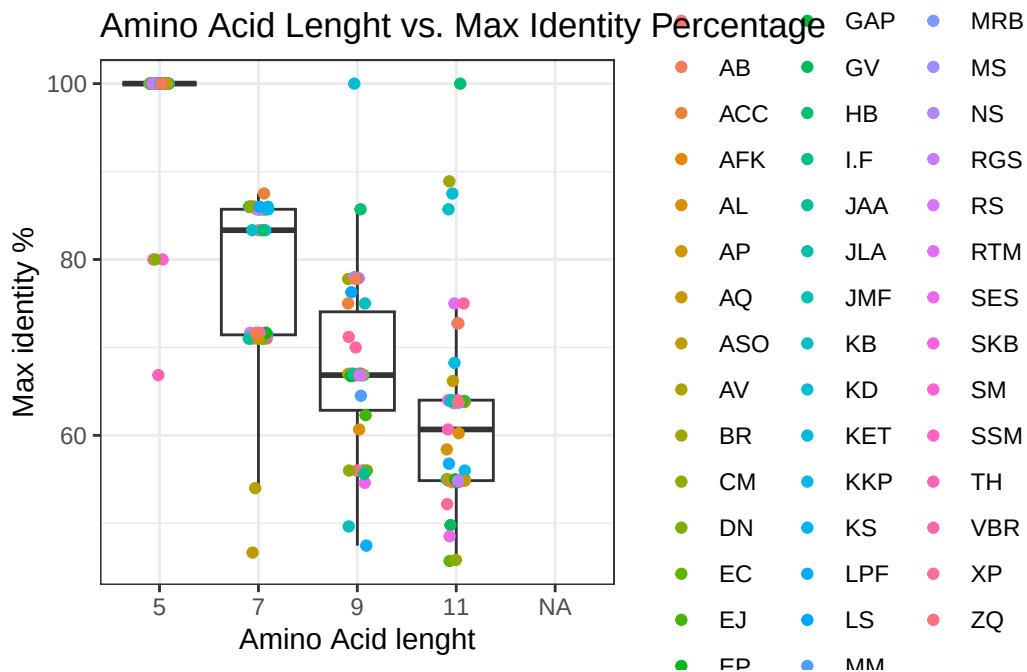


Question 8: Let's add a `geom_jitter()` layer to your box plot to visualize the individual data points on top of the box plot and highlight your own data points. a. First, add a `geom_jitter()` layer to your box plot with each data point colored according to students' initials by adding the following additional line to your plot: `+ geom_jitter(aes(col=Your_Initials))`. Feel free to narrow the spread of the jitter points using the `width` argument (e.g. `width = 0.2`). You can find your own datapoints this way via the color legend, but it looks rather messy. [1 pt]

```
#to the peptide plot object I made of the plot, I added geom jitter and colored the dots based on
peptide_plot + geom_jitter(aes(col= Your_Initials), width = 0.1)
```

Warning: Removed 4 rows containing non-finite outside the scale range (``stat_boxplot()``).

Warning: Removed 4 rows containing missing values or values outside the scale range (``geom_point()``).



Question 8B: Now let's control the coloring ourselves to better highlight your own data points. We can do this by generating a vector with a color for each datapoint (we will use your favorite color for your datapoints and "GREY" for all other datapoints). You can then use this vector to assign colors with the `geom_jitter()` call. i. First, use the `rep()` function to generate a vector consisting of as many repeats of "GREY" as there are datapoints in the `peptide_df` data frame. Assign this vector to an object you name `color`: i.e. `color <- c(rep(,))` (fill in the blanks). Once you have generated the color object, use the `table()` function to ensure that you have the correct number of instances of "GREY" in `color`. [1 pt]

```
rep("GREY",dim(peptide_df)[1])
```

```
[1] "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY"
[11] "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY"
[21] "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY"
[31] "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY"
[41] "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY"
[51] "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY"
[61] "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY"
[71] "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY"
[81] "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY"
```



```
[91] "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY"
[101] "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY"
[111] "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY"
[121] "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY"
[131] "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY"
[141] "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY"
[151] "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY"
[161] "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY" "GREY"
[171] "GREY" "GREY" "GREY" "GREY" "GREY" "GREY"
```

```
#generated a vector with 44 GREY as there are 44 datapoints
color <- c(rep("GREY",dim(peptide_df)[1]))
#assigned the vector an object named color
table(color)
```

```
color
GREY
176
```

```
#checked to make sure I have the correct number of GREY in color
```

8Bii. Next, use the `Your_Initials` column in the `peptide_df` dataframe (which can be called as `peptide_df$Your_Initials`) to generate a **TRUE/FALSE** vector indicating which datapoints are yours (**TRUE**) and which are not (**FALSE**). Assign this vector to an object you name `mydata`: i.e. `mydata <- _____` (fill in the blank). Again, check that you have the correct number of **TRUE** and **FALSE** in `mydata` using the `table()` function.

```
#made an object called mydata and assigned the vector peptide_df$Your_Initial == "MS", which
mydata <- peptide_df$Your_Initial == "MS"

#mydata table to check if the numbers are correct
table(mydata)
```

```
mydata
FALSE  TRUE
172     4
```

Question8Biii: Now, use the `mydata` vector to replace the “**GREY**” values in the `color` vector that correspond to your data points with your

favorite color, like this: `color[mydata] <- "COLOR"` (replace "COLOR" with the name of your favorite color; see link at the bottom of the page to a webpage with 657 R color names to select a cool color). Note, this will replace "GREY" with your favorite color for all the positions that are TRUE in mydata, i.e. the positions of your data points. Again, check the vector with `table()`.

```
#followed the code provided, plugged in my data in the object color, so it picks out my data  
  
color[mydata] <- "palevioletred2"  
  
#checked the number of initials colored pale violet  
table(color[mydata])
```

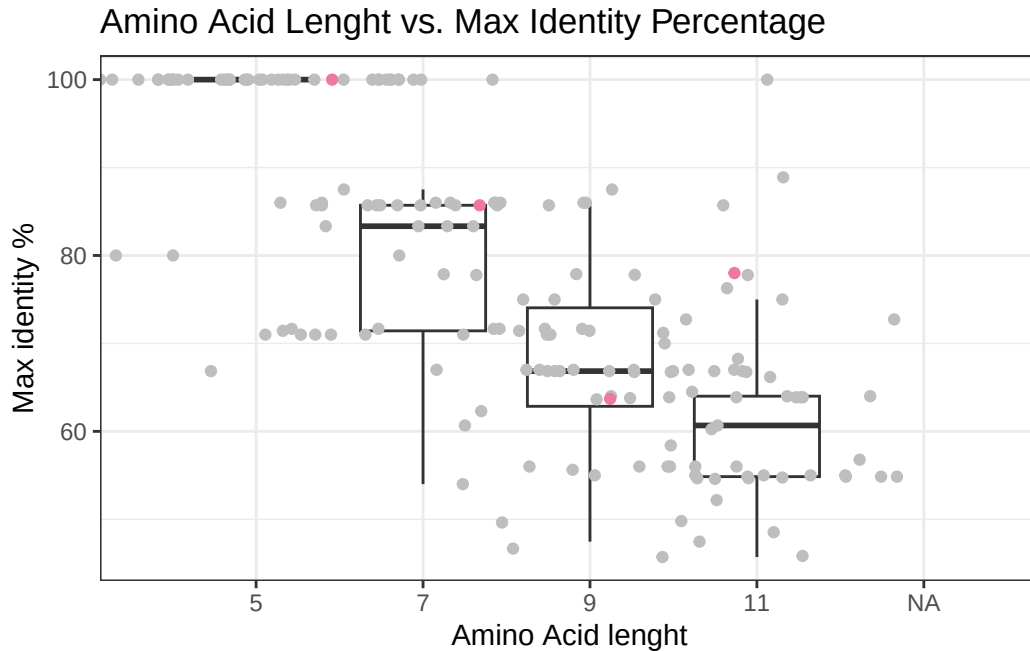
```
palevioletred2  
4
```

Question 8B iv: Now you are ready to remake the plot by adding the following line to your original box plot from Q7: `+ geom_jitter(col=color)`. (Note that the color call is not run as an aesthetic here because we are now directly telling ggplot what color to use for each dot). Again, feel free to use the width argument to narrow the jitter points if you wish (e.g. `width = 0.2`).

```
#used the object i had assigned my plot to, and added geom_jitter onto it.  
#in geom_jitter, specified it to use the object color as the color for the datapoints. also m  
  
peptide_plot + geom_jitter(col=color, width= 1)
```

```
Warning: Removed 4 rows containing non-finite outside the scale range  
(`stat_boxplot()`).
```

```
Warning: Removed 4 rows containing missing values or values outside the scale range  
(`geom_point()`).
```



Question 8c: Based on your boxplot, does any of your own peptide data fall outside the 25 to 75 percentile of the class's data? If yes, list which one(s).

My data point for amino acid length 9 fell outside of the 25 to 75 percentile of the class. However, it is still within the maximum.

Question 9: MHC class II molecules of our immune system display 9 amino acid peptides on the surface of immune cells that, when from a foreign origin, activates a T-cell immune response. Based on your findings in Q7 and 8, speculate why those peptides are 9 amino acids long rather than, say, 5 amino acids.

I suspect that 9 amino acids are the optimal length because it provides just enough identity for the T cells to be activated against foreign pathogens, while also not constantly self-identifying and attacking itself because 5 amino acids tend to provide 100% identity match, meaning there will most likely be a cell in our body that has those amino acids and are not harmful. On the other end of this, 11 amino acids might be too specific for T cells so they only provide protection against a very specific set of foreign pathogens, rather than providing more immunity against more foreign pathogens.